```
version: '3'

services:

  web:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - .:/usr/src/app

  redis:
    image: redis

  database:
    image: postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: some-long-secure-password
      POSTGRES_DB: myapp_development
```

We've defined a new database service using the official postgres image.[1] We're relying on this image's default CMD instruction—CMD ["postgres"]—which starts the Postgres server.[2]

As with redis, our new database service has no need for port mappings or volumes. We don't want the database to be accessible externally to our application, and we don't need to mount any files into the Postgres container.

We do, however, specify a new property called environment. You may well be able to guess what this does: it tells Docker to set the subsequent environment variables inside the container. Here, we're specifying that POSTGRES_USER should be set to postgres; POSTGRES_PASSWORD should be set to some-long-secure-password; and POSTGRES_DB should be set to myapp_development.

Why do we set these?

Just like the non-Dockerized version of Postgres allows you to specify certain parameters as environment variables,[3] the same is true for the Dockerized version.[4] When Postgres launches, if POSTGRES_USER is set, its value will be used as the name of the superuser account. Similarly, if POSTGRES_PASSWORD is set, this will be used as the superuser password. Finally, if POSTGRES_DB is set, this will be used as the name of the default database that's created on launch.

1. https://hub.docker.com/_/postgres/
2. https://github.com/docker-library/postgres/blob/674466e0d47517f4e05ec2025ae996e71b26cae9/10/Dockerfile#L133
3. https://www.postgresql.org/docs/9.1/static/libpq-envars.html
4. https://hub.docker.com/_/postgres/